//make sure that all identifiers meaning constants, lists/arrays and variables using camel case

Age = 14

Age ← 14

First_Name ← “Name”

Last_Name ← “Last Name”

Names_Collected ← INPUT “enter your last name”

Last_Name ← Names_Collected

Equal to - = //in python it would be ==, one = sign in python be the equivalent of ← assigning a piece of data

An algorithm is merely the sequence of steps taken to solve a problem.

Operator	Meaning/purpose	Example	Result
=	Equal – tests if two expressions are identical	IF a=8 IF name$="JANET" IF a=14	True False False
<	Less Than – tests if the first expression is less than the second	IF a<8 IF name$<"JANET" IF a<14	False True True
>	Greater Than – tests if the first expression is greater than the first	IF a>8 IF name$>"JANET" IF a>14	False False False
<>	Not Equal – tests if two expressions are different	IF a<>8 IF name$<>"JANET" IF name$<>"JAMES" IF a<>14	False True False True
<=	Less Than or Equal	IF a<=8 IF name$<="JANET" IF a<=14	True True True
>=	Greater Than or Equal	IF a>=8 IF name$>="JANET" IF a>=14	True False False

Write a program that compares two different numbers. If number 1 is bigger than number two, we want the program to output “number 1 is bigger”, otherwise we want them to enter the number again until number 1 is bigger than number 2.

```
Num1 ← INPUT “Enter a number”//this is how to to setup a variable.
Num2 ← INPUT “Enter a number”
IF Num1 > Number2
        OUTPUT “number 1 is bigger”
ELSE
        OUTPUT “number 1 is not bigger than number 2, try again”
        Num1 ← INPUT “Enter a number”

Name ← INPUT “Enter a programmer name”
IF Name = “Essa” OR “essa” OR “ESSA”
        OUTPUT “great programmer”
IF Name = “Rob” or ELSE
        OUTPUT “terrible programmer”
```

```
IF { conditional statement } THEN
    { statement 1 }
ELSE
    { statement 2 }
ENDIF
```

```
IF {conditional statement} THEN
        OUTPUT {statement 1}
ELSE
        OUTPUT {statement 2}
ENDIF
```

Loops

,

```
FOR count ← 1 TO 5000
        INPUT House “Enter your house value”
        IF House > 50 000 THEN
                Tax← House * 0.010
        ELSE IF House > 100 000 THEN
                Tax ← House * 0.015
        ELSE IF House > 200 000 THEN
                Tax← House * 0.020
        ELSE
                Tax ←  0
                OUTPUT Tax
NEXT
```

```
X ← TRUE
WHILE  X ← TRUE
        INPUT House "Enter your house value"
        IF House > 50 000 THEN
                Tax← House * 0.010
        ELSE IF House > 100 000 THEN
                Tax ← House * 0.015
        ELSE IF House > 200 000 THEN
                Tax← House * 0.020
        ELSE
                Tax ←  0
                OUTPUT Tax
WEND


INPUT X "Enter your value: "

WHILE X != 0 THEN
        N ←  X * X/(1 − X)
        X -= 1
        OUTPUT N
```

To set up an array in pseudocode, you do the following:

Identifier ← [ ]//identifier is name of the array

An alternative method is:

DECLARE Identifier ← [ ]

A 1D array would look like this

DECLARE Names ← [ ]

If you know the size of the array it would be:

DECLARE Names ← [1:30]

A 2D array would look like this:

DECLARE Names ← [[1:30],[1:30]]//arrays begin at index 1 in pseudocode

An array with existing elements would like this:

Names ← [“Jeet”, “Jord”, “Jeremy”]

To add a new name to the array, you would use one of the following:
Name ← USERINPUT “Enter your name”
Names ← Names + Name //add the variable after the array name

Another method you could use is:

Names.APPEND(Name)